

Document Processing & Automation

Architecture & Implementation Spec

Stack: Next.js + Tailwind CSS (frontend), FastAPI
(backend)

Cloud: Microsoft Azure

Date: July 30, 2026

Author: Generated by ChatGPT

Table of Contents

1. Executive Summary

2. Goals & Success Metrics

3. Architecture Overview (Logical)

4. End-to-End Data Flow & Sequence

5. Components & Azure Services Mapping

6. Data Models & Schema

7. API Contract (Minimum)

8. Human Review UI Features

9. Business Rules & Validation (Examples)

10. Security & Compliance

11. Network & Private Endpoints

12. Scalability & Resilience

13. Observability & Monitoring

14. CI/CD & Deployment

15. Cost & Sizing Guidance

16. Error Handling & Retries

17. Testing Strategy

18. Deployment Checklist (First-Run)

19. MVP Plan & Timeline

20. Future Enhancements

21. Risks & Mitigations

Appendices

A. PlantUML Sources (Diagrams)

B. FastAPI Code Snippets

C. Next.js Upload & Review Snippets

D. GitHub Actions CI/CD Outline

E. Starter Bicep Infra Template

1. Executive Summary

Build a scalable, secure Document Processing & Automation platform for ingesting invoices, receipts, contracts, and other documents; extracting structured data; supporting human review; indexing for search; and integrating with downstream systems. The stack uses Next.js + Tailwind for the UI and FastAPI for the backend, deployed to Azure services.

2. Goals & Success Metrics

Goals:

- Automated extraction and classification
- Human-in-the-loop review for low-confidence results
- Searchable archive with metadata
- Integration to ERP/DMS
- Strong audit & compliance capabilities

Success Metrics:

- Documents processed per day
- % auto-validated (no human review)
- Avg processing latency
- Field-level extraction accuracy
- Manual review time saved
- Cost per document

3. Architecture Overview (Logical)

High-Level Components:

- Next.js frontend (Azure Static Web Apps)
- FastAPI backend (App Service / AKS)
- Blob Storage (incoming / processed / archive)
- Event Grid → Service Bus
- Workers (Azure Functions / AKS) for processing
- Azure Form Recognizer for OCR/extraction
- Azure Cognitive Search for indexing & search
- Cosmos DB (metadata & audit)
- Key Vault, Azure AD / B2C, Application Insights

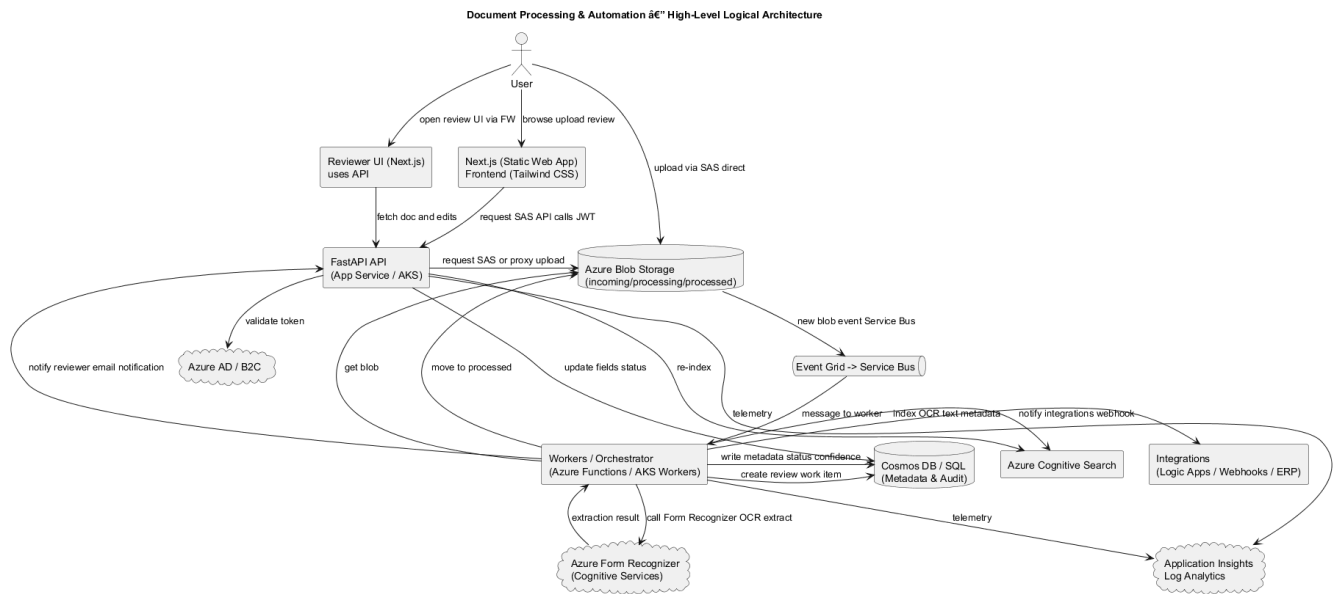


Figure 1: High-Level Logical Architecture

4. End-to-End Data Flow & Sequence

Sequence (Summary):

1. Client requests SAS or uploads via FastAPI.
2. Blob Storage receives file → Event Grid publishes → Service Bus enqueues message.
3. Worker downloads blob, preprocesses, calls Form Recognizer for extraction.
4. Worker validates; auto-accept or create review task.
5. On accept: persist to DB, index to Search, move blob to processed, integrate downstream.
6. On review: reviewer edits via Next.js UI; FastAPI updates DB & search; then integration.
7. Audit trail appended.

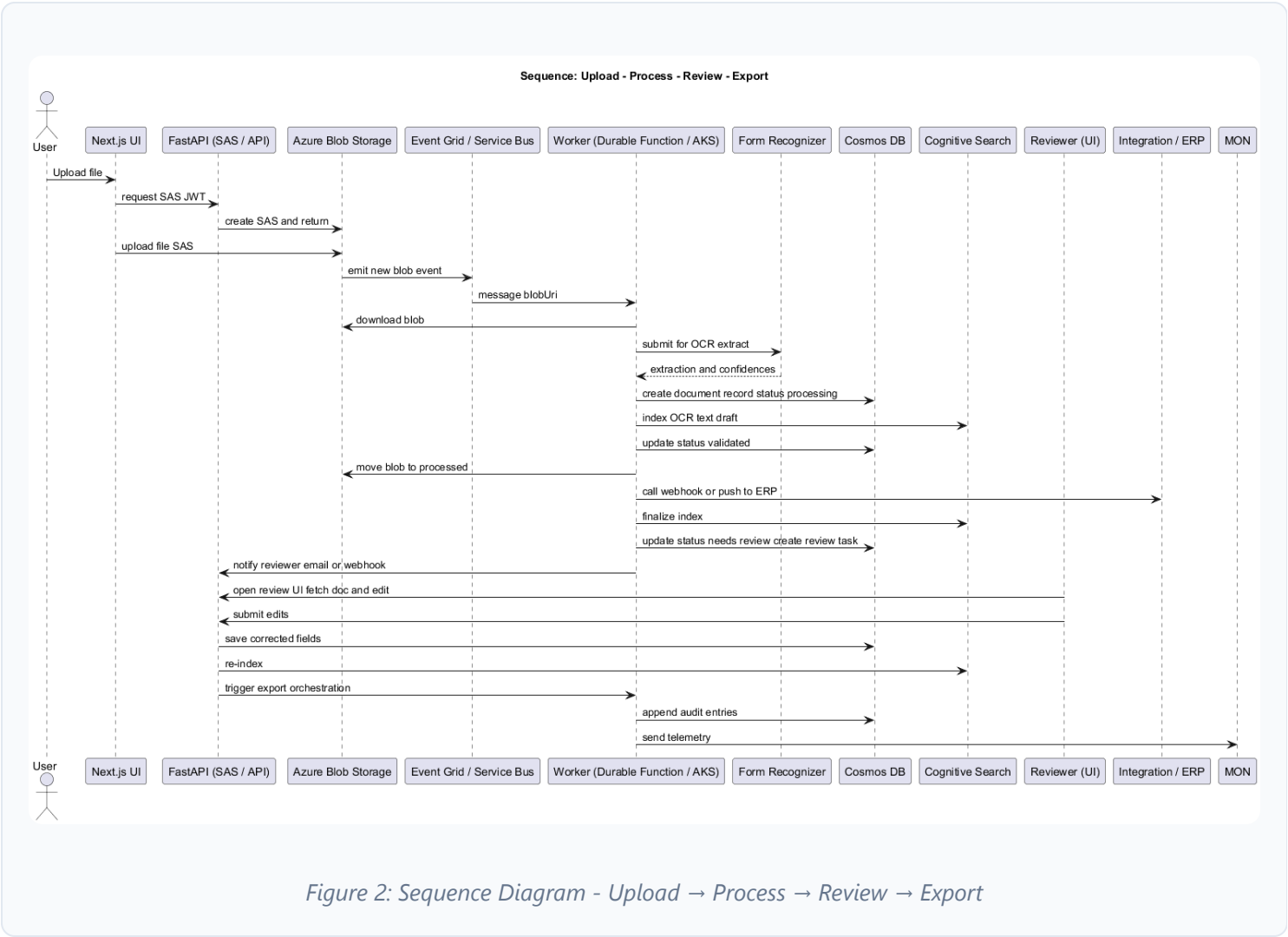


Figure 2: Sequence Diagram - Upload → Process → Review → Export

5. Components & Azure Services Mapping

Component	Azure Service
Frontend	Azure Static Web Apps (Next.js)
API	FastAPI container → App Service for Containers / AKS
Storage	Azure Blob Storage (containers)
Eventing	Event Grid → Service Bus
Processing	Durable Functions OR AKS workers
OCR	Azure Form Recognizer (Cognitive Services)
Search	Azure Cognitive Search
Metadata	Azure Cosmos DB
Secrets	Azure Key Vault
Auth	Azure AD / B2C
Monitoring	Application Insights + Log Analytics
Integrations	Logic Apps, webhooks, Power Automate

6. Data Models & Schema

Document Record (JSON):

```
{
  "id": "guid",
  "tenantId": "string",
  "blobUri": "https://...",
  "filename": "string",
  "contentType": "application/pdf",
  "documentType": "invoice|receipt|contract|other",
  "extractedFields": {
    "vendorName": "string",
    "invoiceNumber": "string",
    "invoiceDate": "YYYY-MM-DD",
    "totalAmount": 1234.56,
    "currency": "USD",
    "lineItems": [
      {
        "description": "",
        "qty": 1,
        "unitPrice": 12.34,
        "amount": 12.34
      }
    ]
  },
  "confidenceScores": {
    "vendorName": 0.98,
    "totalAmount": 0.95
  },
  "status": "ingested|processing|needs_review|validated|exported|archived",
  "createdBy": "userId",
  "createdAt": "2026-07-30T12:00:00Z",
  "audit": [
    {
      "actor": "userId",
      "action": "ingest",
      "timestamp": "..."
    }
  ]
}
```

Search Index Fields:

- id, searchableText (OCR), documentType, vendorName, invoiceDate (date), totalAmount (number), status

7. API Contract (Minimum)

- `POST /api/documents/upload` — returns: { documentId, uploadSasUrl } or accepts file
- `GET /api/documents/{id}` — returns: metadata + extractedFields
- `GET /api/documents/{id}/download` — returns: presigned SAS URL
- `GET /api/documents?query=...` — returns: search results (proxy to Cognitive Search)
- `GET /api/review/queue?filters=...`
- `POST /api/review/{id}/submit` — body: { correctedFields, reviewerId, comments }
- `POST /api/export/{id}/trigger`
- `POST /api/webhook/inbound`

8. Human Review UI Features

- Queue with filters (confidence threshold, doc type, age)
- PDF/Image viewer with field bounding box highlights
- Editable extracted fields
- Batch actions: approve, reject, assign
- Audit timeline per document
- Role-based access

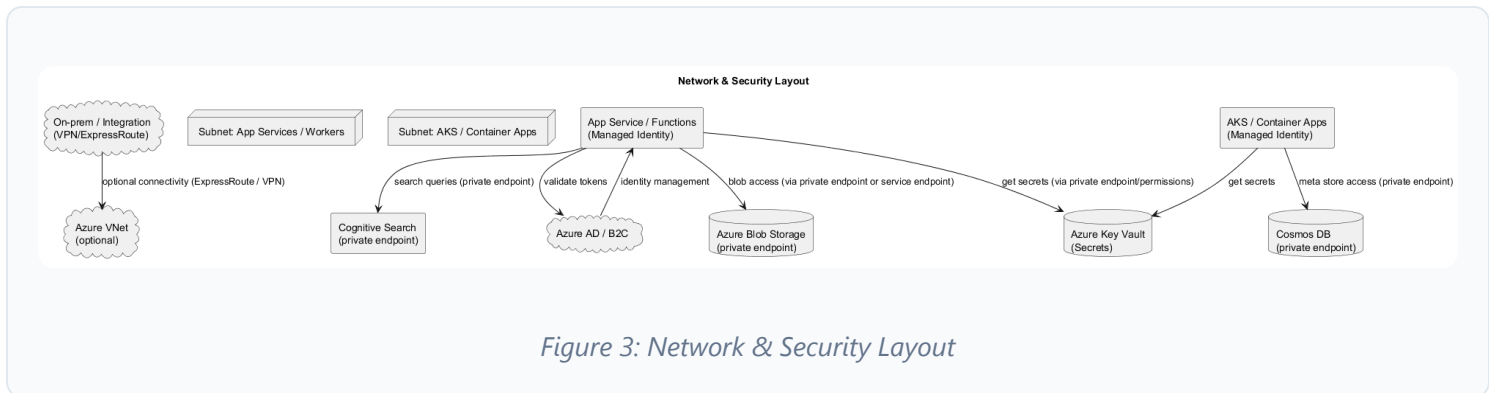
9. Business Rules & Validation (Examples)

- invoiceDate within last 2 years
- totalAmount equals sum(lineItems) within tolerance
- vendor fuzzy-match against master vendor DB
- detect duplicates by vendor + invoiceNumber + date

10. Security & Compliance

- **Authentication:** Azure AD / B2C; JWT validation in FastAPI
- **Authorization:** RBAC, tenant-scoped access
- **Secrets:** Key Vault + Managed Identities
- **Encryption:** TLS in transit; SSE for Storage; TDE for Cosmos DB
- **Network:** Private Endpoints for Storage, Cosmos, Search (optional)
- **Audit:** append-only audit trail; blob immutability if required
- **Data Residency:** choose region(s) and disable geo-replication unless compliant

11. Network & Private Endpoints



12. Scalability & Resilience

- Decouple via Service Bus for bursts
- Serverless (Functions) for elastic scaling or AKS with HPA for steady high volume
- Dead-letter queue pattern
- Durable Functions for orchestrations and checkpointing
- Cosmos DB partitioning & autoscale
- Cognitive Services tier selection and retry/backoff

13. Observability & Monitoring

- Application Insights (FastAPI, Functions)
- Log Analytics for logs & queries
- Metrics to track: queue depth, throughput, avg confidence, review backlog
- Alerts: queue > threshold, function failures, low average confidence, API 5xx surge
- Dashboards in Azure Monitor / Power BI

14. CI/CD & Deployment

- **Repo:** GitHub (Next.js, FastAPI, IaC)
- **Build:** GitHub Actions run tests, build Docker image, push to Azure Container Registry
- **Deploy:**
 - Next.js → Azure Static Web Apps (automatic GitHub Actions)
 - FastAPI → App Service for Containers or AKS (image from ACR)
 - Infra → ARM/Bicep/Terraform via GitHub Actions
- **Runtime:** Use Managed Identity at runtime; Key Vault for secrets

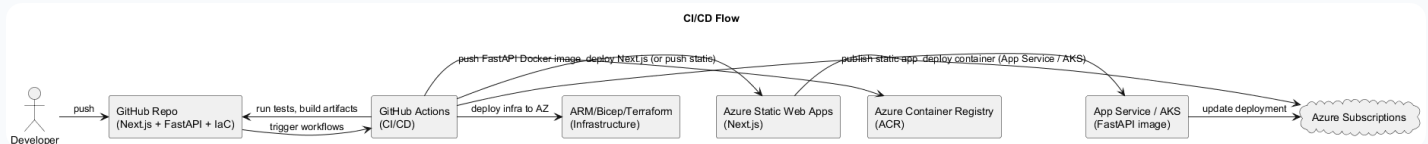


Figure 4: CI/CD Flow

15. Cost & Sizing Guidance

Pilot:

Static Web Apps free, Functions consumption, Form Recognizer S0, small Cosmos DB

Production:

Functions Premium or AKS, Form Recognizer S1+, Service Bus Premium, Cosmos DB autoscale, Cognitive Search Standard

Optimizations:

Direct-to-blob upload, Cool/Archive tiers, batch non-urgent processing

16. Error Handling & Retries

- Dead-letter queue after N retries
- Exponential backoff for transient errors
- Manual retry UI for operators
- Durable Functions with checkpoints for resume

17. Testing Strategy

- Unit tests for parsing & business rules
- Integration tests with mocked Form Recognizer & Search
- E2E tests with sample doc set
- Performance tests for peak bursts

18. Deployment Checklist (First-Run)

- Create resource group, Storage account, Form Recognizer resource, Service Bus, Cosmos DB, App Service / AKS, Key Vault, Static Web App
- Configure Managed Identities & RBAC
- Set up Private Endpoints if required
- Implement SAS issuance endpoint in FastAPI
- Add Application Insights instrumentation
- Run end-to-end tests with sample docs

19. MVP Plan & Timeline

MVP Scope (6–10 weeks):

- Upload (direct-to-blob via SAS)
- Form Recognizer extraction (invoices/receipts)
- Persist to Cosmos DB, index to Cognitive Search
- Basic review UI for low-confidence docs
- Export to SFTP/CSV

Team:

- 1 backend (4–6 weeks)
- 1 frontend (3–5 weeks)
- 1 ML/data engineer (part-time 2–4 weeks)
- 1 QA/devops (part-time)

20. Future Enhancements

- Active learning & retraining
- Contract clause extraction via custom models
- PII redaction
- Multi-language extraction
- Metering & tenant billing

21. Risks & Mitigations

Risk	Mitigation
Poor scan quality	Image pre-processing and manual review
Model drift	Feedback loop and scheduled retraining
Sensitive data leakage	Strict RBAC, private endpoints, encryption

Appendix A — PlantUML Sources for Diagrams

Diagram 1 — High-Level Logical Architecture

```
@startuml
title Document Processing & Automation — High-Level Logical Architecture

actor User
rectangle "Next.js (Static Web App)\nFrontend (Tailwind CSS)" as FW
rectangle "FastAPI API\n(App Service / AKS)" as API
database "Azure Blob Storage\n(incoming/processing/processed)" as STORAGE
queue "Event Grid -> Service Bus" as BUS
rectangle "Workers / Orchestrator\n(Azure Functions / AKS Workers)" as WORK
cloud "Azure Form Recognizer\n(Cognitive Services)" as FR
rectangle "Azure Cognitive Search" as SEARCH
database "Cosmos DB / SQL\n(Metadata & Audit)" as DB
rectangle "Reviewer UI (Next.js)\nuses API" as REVIEW
cloud "Azure AD / B2C" as AUTH
rectangle "Integrations\n(Logic Apps / Webhooks / ERP)" as INT
cloud "Application Insights\nLog Analytics" as MON

User --> FW : browse / upload / review
FW --> API : request SAS / API calls (JWT)
API --> AUTH : validate token
API -> STORAGE : request SAS or proxy upload
User -> STORAGE : upload via SAS (direct)
STORAGE --> BUS : new blob event -> Service Bus
BUS --> WORK : message to worker
WORK -> STORAGE : get blob
WORK -> FR : call Form Recognizer (OCR & extract)
FR --> WORK : extraction result
WORK -> DB : write metadata, status, confidence
WORK -> SEARCH : index OCR text + metadata
WORK -> STORAGE : move to processed
WORK -> INT : notify integrations / webhook
WORK -> DB : create review work item
WORK -> API : notify reviewer via email/notification
User --> REVIEW : open review UI (via FW)
REVIEW --> API : fetch doc & edits
API -> DB : update fields / status
API -> SEARCH : re-index
WORK --> MON : telemetry
API --> MON : telemetry
@enduml
```

Diagram 2 — Sequence: Upload -> Process -> Review -> Export

```
@startuml
title Sequence: Upload -> Process -> Review -> Export
```

```

actor User
participant "Next.js UI" as UI
participant "FastAPI (SAS / API)" as API
participant "Azure Blob Storage" as BLOB
participant "Event Grid / Service Bus" as BUS
participant "Worker (Durable Function / AKS)" as WORK
participant "Form Recognizer" as FR
participant "Cosmos DB" as DB
participant "Cognitive Search" as SEARCH
participant "Reviewer (UI)" as REVIEW
participant "Integration / ERP" as INT

User -> UI : Upload file
UI -> API : request SAS (JWT)
API -> BLOB : create SAS / return
UI -> BLOB : upload file (SAS)
BLOB -> BUS : emit new blob event
BUS -> WORK : message (blobUri)
WORK -> BLOB : download blob
WORK -> FR : submit for OCR/extract
FR --> WORK : extraction + confidences
WORK -> DB : create document record (status=processing)
WORK -> SEARCH : index OCR text (draft)
WORK -> DB : update status=validated
WORK -> BLOB : move blob -> processed
WORK -> INT : call webhook / push to ERP
WORK -> SEARCH : finalize index
WORK -> DB : update status=needs_review, create review task
WORK -> API : notify reviewer (email/webhook)
REVIEW -> API : open review UI, fetch doc & edit
REVIEW -> API : submit edits
API -> DB : save corrected fields
API -> SEARCH : re-index
API -> WORK : trigger export/orchestration
WORK -> DB : append audit entries
WORK -> MON : send telemetry
@enduml

```

Diagram 3 — Network & Security Layout

```

@startuml
title Network & Security Layout

cloud "Azure VNet\n(optional)" as VNET
node "Subnet: App Services / Workers" as SUB_APPS
node "Subnet: AKS / Container Apps" as SUB_AKS

rectangle "App Service / Functions\n(Managed Identity)" as APPS
rectangle "AKS / Container Apps\n(Managed Identity)" as K8S
database "Azure Key Vault\n(Secrets)" as KV
database "Azure Blob Storage\n(private endpoint)" as PST

```



```

database "Cosmos DB\n(private endpoint)" as COS
rectangle "Cognitive Search\n(private endpoint)" as CS
cloud "Azure AD / B2C" as AUTH
cloud "On-prem / Integration\n(VPN/ExpressRoute)" as ONPREM

AUTH --> APPS : identity management
APPS --> KV : get secrets (via private endpoint/permissions)
K8S --> KV : get secrets
APPS --> PST : blob access (via private endpoint or service endpoint)
K8S --> COS : meta store access (private endpoint)
APPS --> CS : search queries (private endpoint)
ONPREM --> VNET : optional connectivity (ExpressRoute / VPN)
APPS --> AUTH : validate tokens
@enduml

```

Diagram 4 — CI/CD Flow

```

@startuml
title CI/CD Flow

actor Developer

rectangle "GitHub Repo\n(Next.js + FastAPI + IaC)" as REPO
rectangle "GitHub Actions\n(CI/CD)" as GHA
rectangle "Azure Container Registry\n(ACR)" as ACR
rectangle "Azure Static Web Apps\n(Next.js)" as SWA
rectangle "App Service / AKS\n(FastAPI image)" as APPS
rectangle "ARM/Bicep/Terraform\n(Infrastructure)" as IAC
cloud "Azure Subscriptions" as AZ

Developer -> REPO : push
REPO -> GHA : trigger workflows
GHA -> REPO : run tests, build artifacts
GHA -> ACR : push FastAPI Docker image
GHA -> SWA : deploy Next.js (or push static)
GHA -> IAC : deploy infra to AZ
GHA -> APPS : deploy container (App Service / AKS)
APPS -> AZ : update deployment
SWA -> AZ : publish static app
@enduml

```

Appendix B — FastAPI Code Snippets

```

# snippets/fastapi_sas_and_auth.py (example)
from fastapi import FastAPI, HTTPException
from azure.identity import DefaultAzureCredential
from azure.storage.blob import BlobServiceClient
import os, jwt, requests

app = FastAPI()

```

```

def verify_jwt(token: str):
    # Production: validate signature using JWKS; this is illustrative only
    try:
        payload = jwt.decode(token, options={"verify_signature": False})
        return payload
    except Exception:
        raise HTTPException(status_code=401, detail="Invalid token")

@app.post("/api/documents/upload")
async def create_upload_sas(token: str, filename: str):
    payload = verify_jwt(token)
    account_url = os.environ["AZURE_STORAGE_ACCOUNT_URL"]
    credential = DefaultAzureCredential()
    blob_service = BlobServiceClient(account_url=account_url, credential=credential)
    container_client = blob_service.get_container_client("incoming")
    blob_client = container_client.get_blob_client(filename)
    # Implement user delegation SAS in prod; placeholder below:
    sas_url = f"{account_url}/incoming/{filename}?sasToken=REPLACE_ME"
    return {"documentId": "guid-placeholder", "uploadSasUrl": sas_url}

def call_form_recognizer(endpoint, key, blob_sas_url):
    headers = {"Ocp-Apim-Subscription-Key": key}
    analyze_url = f"{endpoint}/formrecognizer/documentModels/prebuilt-invoice/analyze?api-version=2024-06-01"
    r = requests.post(analyze_url, headers=headers, json={"url": blob_sas_url})
    r.raise_for_status()
    return r.json()

```

Appendix C — Next.js Upload & Review Snippets (Tailwind)

Upload Page

```

// frontend/pages/upload.js
import { useState } from "react";
export default function Upload() {
    const [file, setFile] = useState(null);
    async function handleUpload(e) {
        e.preventDefault();
        const filename = file.name;
        const res = await fetch(`/api/sas?filename=${encodeURIComponent(filename)}`);
        const data = await res.json();
        const sasUrl = data.uploadSasUrl;
        await fetch(sasUrl, {
            method: "PUT",
            headers: { "x-ms-blob-type": "BlockBlob", "Content-Type": file.type },
            body: file
        });
        alert("Uploaded");
    }
}

```

```

return (
  <form onSubmit={handleUpload} className="p-4">
    <input type="file" onChange={e => setFile(e.target.files[0])} />
    <button className="bg-blue-600 text-white px-4 py-2 rounded">Upload</button>
  </form>
);
}

```

Review Page

```

// frontend/pages/review/[id].js
import { useState, useEffect } from "react";
import { useRouter } from "next/router";
export default function ReviewPage() {
  const router = useRouter();
  const { id } = router.query;
  const [doc, setDoc] = useState(null);
  useEffect(() => {
    if (!id) return;
    fetch(`/api/documents/${id}`).then(r => r.json()).then(setDoc);
  }, [id]);
  if (!doc) return <div>Loading...</div>;
  return (
    <div className="p-4">
      <h1 className="text-xl font-bold">Review {doc.filename}</h1>
      <div className="mt-4">
        <img src={doc.previewUrl} alt="preview" className="border" />
      </div>
      <form>
        <label className="block mt-2">Vendor</label>
        <input defaultValue={doc.extractedFields.vendorName} className="border p-2" />
      </form>
    </div>
  );
}

```

Appendix D — GitHub Actions CI/CD Outline

```

# .github/workflows/ci-cd.yml (outline)
name: CI-CD
on:
  push:
    branches: [ main ]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Set up Python

```

```

    uses: actions/setup-python@v4
    with: { python-version: "3.11" }
  - name: Install backend deps
    run: pip install -r backend/requirements.txt
  - name: Run backend tests
    run: pytest backend/tests
build-and-deploy:
  needs: test
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - name: Build and push backend image
      uses: docker/build-push-action@v4
      with:
        push: true
        tags: ${ secrets.ACR_LOGIN_SERVER }/docproc-fastapi:${ github.sha }
    - name: Deploy infra (Bicep/Terraform)
      run: |
        # az deployment group create ... or terraform apply
    - name: Deploy FastAPI (App Service)
      run: |
        # az webapp up ... or kubectl rollout
    - name: Deploy Next.js (Static Web Apps)
      uses: Azure/static-web-apps-deploy@v1
      with:
        # static web apps config

```

Appendix E — Starter Bicep Snippet (Core Infra)

```

// infra/starter.bicep
param location string = resourceGroup().location
param storageAccountName string
resource storage 'Microsoft.Storage/storageAccounts@2023-05-01' = {
  name: storageAccountName
  location: location
  sku: { name: 'Standard_LRS' }
  kind: 'StorageV2'
  properties: { accessTier: 'Hot' }
}
output storageAccountId string = storage.id

```